Rapport d'activités

24/11/2025

Tom Planche

2025-2026

┌┬┬┐
││└ Remerciements 3
│└ Introduction 4
└ Environnement de Travail 5
  ├ Conditions de travail et intégration 5
  │ ├ Environnement technique et outils 5
  │ └ Organisation du travail en mode agile 7
  └ Conclusion partielle 9
└ Missions 11
  └ Optimisation de la requête `getAttendanceStatsForAPeriod` 11
    ├ Contexte et problématique 11
    ├ Analyse technique de la cause racine 11
    ├ Solution architecturale 11
    ├ Modifications techniques implémentées 12
    ├ Résultats et impact 13
    ├ Explication des optimisations clés 13
    ├ Impact en production 14
    ├ Conformité aux patterns existants 14
    └ Enseignements techniques 14

Chapter 1

Remerciements

Chapter 2

Introduction

Ce rapport dresse le bilan de mon expérience en entreprise au cours de l'année scolaire 2025-2026. Ce parcours s'inscrit dans le cadre de ma formation d'ingénieur en informatique et systèmes d'information, réalisée en alternance au sein de l'École d'Ingénieur du Conservatoire National des Arts et Métiers (Ei-CNAM)

J'ai donc intégré en alternance l'équipe backend d'Affluences, une entreprise française innovante spécialisée dans la gestion de l'affluence et l'optimisation des flux de visiteurs. J'intègre la partie **Internal Services** en tant que **développeur backend**.

Chapter 3

Environnement de Travail

I - Conditions de travail et intégration

I.I - Environnement technique et outils

L'environnement technique chez Affluences repose sur une infrastructure moderne et des outils soigneusement sélectionnés pour faciliter le développement et la collaboration.

I.I.1 - Stack technique principale

L'entreprise utilise un ensemble d'outils standardisés pour tous les développeurs :

IDE : Les développeurs peuvent choisir leur environnement (VS Code, WebStorm, ou autres)
Gestion de versions : Git avec GitLab comme plateforme de dépôt
Conteneurisation : Docker et Docker Compose pour l'isolation des environnements
Runtime JavaScript : Node.js avec npm, avec possibilité d'utiliser des gestionnaires de version (nvm, nodev)
Base de données : MySQL (production et staging), MongoDB, Redis/Valkey
Messagerie : Kafka pour les systèmes événementiels

I.I.2 - Langages et frameworks

La stack technique privilégie certains langages et frameworks pour garantir la cohérence entre les projets :

- **Backend** : Node.js, TypeScript, Python
- **Frameworks** : TypeORM pour les ORM
- **Formats de logs** : logfmt (format structuré obligatoire)

I.I.3 - Infrastructure et déploiement

L'infrastructure d'Affluences est organisée autour de plusieurs environnements distincts :

Definition 3.1. (Environnements):
- **Staging** : environnement de pré-production accessible via les URLs en `.dev` ou `.io`
- **Production** : environnement client accessible via les URLs en `.com` ou `.live`

I.I.4 - Outils de productivité

Pour la visualisation et les tests, l'entreprise met à disposition :

- **Postman** et/ou **Swagger UI** : pour tester les services et la documentation des APIs
- **Workbench ou DataGrip** : pour la visualisation des bases de données
- **Portainer** : gestionnaire d'images Docker où les environnements sont déployés

I.I.5 - Sécurité et accès aux ressources

L'accès aux ressources internes est sécurisé par plusieurs mécanismes :

Cloudflare Access et Tunnels

Affluences utilise Cloudflare Access comme reverse-proxy authentifié pour protéger les services privés. L'authentification se fait via les comptes Google Affluences. Cette solution permet d'accéder aux bases de données et APIs internes sans VPN traditionnel.

Pour les connexions aux environnements distants, l'entreprise utilise également **Fast Travel**, une solution VPN interne basée sur Wireguard, avec deux réseaux distincts :

- Réseau de production : `prod.affluences.network` (172.30.0.0/16)
- Réseau de staging : `staging.affluences.network` (172.31.0.0/16)

I.I.6 - Observabilité et monitoring

L'infrastructure de monitoring comprend :

- **Datadog** : plateforme centralisée d'agrégation et d'analyse des logs (région EU)
- **Prometheus** : collecte des métriques applicatives
- **Open Telemetry** : pour le tracing distribué (en cours de déploiement)
- **Kafka-UI** : interface de visualisation des topics Kafka

Example 3.1. (Accès aux outils de monitoring): Les outils nécessitent une authentification via Cloudflare Access :

- Staging : `https://kafka-ui.affluences.dev/`
- Production : `https://kafka-ui.affluences.live/`

I.I.7 - Outils collaboratifs

L'équipe utilise une suite d'outils pour la communication et la documentation :

- **GitLab** : hébergement du code et CI/CD
- **Jira** : gestion de projet en mode Agile
- **Gmail/Chat** : communication interne
- **Confluence, Whimsical, KB/KB tech** : documentation
- **Portail Affluences** : portail interne centralisant tous les outils et informations

I.I.8 - Gestion des déploiements

Le déploiement suit une méthodologie rigoureuse avec :

- **ArgoCD** : pour le déploiement continu
  - Staging : `https://cd.affluences.dev/`
  - Production : `https://cd.affluences.live/`
- **Registry GitLab** : stockage des images Docker
- **Portainer** : gestion des conteneurs en production

I.II - Organisation du travail en mode agile

I.II.1 - Méthodologie de développement

Affluences applique une méthodologie Agile structurée avec des étapes de développement clairement définies.

Definition 3.2. (Workflow de développement):

1. Création d'une branche à partir de `staging`
2. Commits avec référence au ticket Jira (format `DP-XXX : description`)
3. Phase de modification avec commits et push réguliers
4. Après vérification, merge sur `staging`

I.II.2 - Pipeline CI/CD

Le processus d'intégration et de déploiement continu suit des étapes standardisées :

Pipeline Staging

1. Compilation
2. Tests automatisés
3. Publication sur le registry
4. Déploiement automatique

Pipeline Production

1. Compilation
2. Tests automatisés
3. Publication sur le registry
4. Déploiement manuel après validation

I.II.3 - Versionnage sémantique

L'entreprise applique le standard Semantic Versioning 2.0.0 pour tous ses projets :

Example 3.2. (Convention de nommage des versions):

- Format : `x.y.z` où :
  - `x` : refonte majeure du projet
  - `y` : changement de structure
  - `z` : fonctionnalité mineure
- Tags pour staging : `release-x.y.z-rc.w` (release candidate)
- Tags pour production : `release-x.y.z` (version stable)

Exemple : `release-1.8.3-rc.0` pour staging, `release-1.8.3` pour production

I.II.4 - Architecture orientée services

Affluences suit une architecture microservices avec des principes clairs :

Definition 3.3. (Principes architecturaux):

- Chaque service est **stateless** et **indépendant**
- Un service est responsable de sa propre base de données
- Les services ne partagent pas de bases de données
- Communication entre services via REST ou GraphQL
- Approche événementielle avec Kafka

L'architecture cible sépare trois couches :

1. **Applications** : clients publics (APIs, web apps) ou internes (algorithmes, scrapers)
2. **Services** : interface entre applications et sources de données
3. **Sources de données** : bases de données et caches

I.II.5 - Standards de qualité

L'équipe applique des standards stricts pour maintenir la qualité du code :

Logging

- Format obligatoire : logfmt (structuré)
- Logs en une seule ligne
- Timestamps en UTC ISO 8601 avec précision milliseconde
- Niveaux : debug, info, notice, warning, error, critical
- Envoi automatique vers Datadog

Variables d'environnement

- Nommage en UPPER_CASE avec underscores
- Préfixes pour grouper les variables liées
- Unités explicites dans les noms (ex: `_SECONDS`, `_MILLISECONDS`)
- Feature toggles préfixés par `ENABLE_`

I.II.6 - Sécurité Docker

Des guidelines strictes de sécurité sont appliquées pour les conteneurs :

- Utilisation d'images de base minimales (Alpine)
- Multi-stage builds obligatoires
- Utilisateurs non-root pour l'exécution
- Scan régulier des vulnérabilités (Snyk, Hadolint)
- Tags de version fixes (pas de `:latest`)
- Fichiers `.dockerignore` pour exclure les données sensibles

I.II.7 - Onboarding et documentation

Le processus d'intégration est structuré avec un guide détaillé pour les nouveaux développeurs :

Example 3.3. (Guide de démarrage rapide): Le Quick Start Guide couvre :
- Installation et configuration de l'environnement
- Accès aux différents services (GitLab, Jira, etc.)
- Configuration SSH et registres
- Mise en place des tunnels Cloudflare
- Lancement des projets en local ou via Docker

I.II.8 - Communication et collaboration

La communication au sein de l'équipe s'organise autour de plusieurs canaux :

- **Chat** : canaux thématiques (#Technical Team, #Internal Services)
- **Jira** : suivi des tickets et sprints
- **GitLab** : revues de code via merge requests
- **Documentation** : Confluence et KB pour la documentation technique

Cette organisation permet une collaboration efficace tout en maintenant la traçabilité des décisions et des modifications.

II - Conclusion partielle

L'environnement de travail chez Affluences se caractérise par une infrastructure technique moderne et des processus bien définis. L'utilisation d'outils standardisés et de méthodologies éprouvées facilite

l'intégration des nouveaux développeurs et assure la qualité des livrables. L'approche microservices couplée à une forte culture DevOps permet à l'équipe de maintenir et faire évoluer efficacement une infrastructure complexe au service de nombreux clients.

Chapter 4

Missions

I - Optimisation de la requête `getAttendanceStatsForAPeriod`

I.I - Contexte et problématique

La requête GraphQL `getAttendanceStatsForAPeriod` présentait des problèmes de performance critiques pour les plages de dates supérieures à un mois. Les requêtes prenaient plus de 90 secondes pour des périodes de 30 jours et crashaient complètement pour des requêtes sur une année entière.

Symptômes observés :

- Requête 30 jours : **90+ secondes**
- Requête 1 an : **Timeout** (crash complet)
- Dégradation exponentielle avec l'augmentation de la période
- Impact négatif sur l'expérience utilisateur des dashboards

I.II - Analyse technique de la cause racine

Le problème résidait dans l'architecture des requêtes du `AttendanceStatsRepository`, qui filtraient les données par `site_id` en utilisant l'extraction JSON :

```
WHERE JSON_EXTRACT(h.contextual_data, "$.site_id") = :siteId
  AND h.record_datetime_utc BETWEEN :from AND :to
  AND JSON_EXTRACT(h.data_points, "$.occupancy") IS NOT NULL
```

Definition 4.4. (Limitations de l'approche initiale):

- Impossibilité d'utiliser l'index de clé primaire `(measuring_set_id, record_datetime_utc)`
- Nécessité d'un parcours complet de la table (**full table scan**)
- Parsing JSON pour chaque ligne de la table
- Performance dégradant de manière exponentielle avec la taille de la période

I.III - Solution architecturale

L'optimisation a consisté à inverser la stratégie de requêtage pour exploiter l'indexation existante de la base de données.

Principe de la solution :

Approche initiale (lente) :
`site_id → [Scan complet + parsing JSON] → Résultats`

Nouvelle approche (optimisée) :
`site_id → [Appel API] → measuring_set_ids → [Requête indexée] → Résultats`

Au lieu de requêter directement par `site_id` (stocké dans un champ JSON), la solution procède en deux étapes :

1. **Récupération des measuring set IDs** via le `SensorsInternalHttpRepository`
2. **Requête par `measuring_set_id`** (colonne indexée) au lieu de `site_id` (champ JSON)

Cette approche ajoute un appel API léger (10-20ms) mais transforme la requête base de données de O(n) en O(log n).

I.IV - Modifications techniques implémentées

I.IV.1 - Refactoring du contrôleur

Le `AttendanceStatsController` a été rendu injectable avec injection de dépendances :

```typescript
@injectable()
export class AttendanceStatsController {
    constructor(private sensorsRepository: SensorsInternalRepository) {}

    private async getMeasuringSetIdsForSite(siteId: number): Promise<string[]> {
        const measuringSets = await this.sensorsRepository.getMeasuringSets({
            siteIds: [siteId],
            typeIn: ['IN_OUT_OCCUPANCY', 'ENTITY_COUNT_OCCUPANCY', 'WAITING_TIME'],
        });
        return measuringSets.map(ms => ms.measuringSetId);
    }
}
```

I.IV.2 - Optimisation des requêtes repository

Les signatures de méthodes ont été modifiées pour accepter des `measuring_set_ids` :

```typescript
public static async getMinMaxOccupancy(
    measuringSetIds: string[],
    fromDatetimeUtc: string,
    toDatetimeUtc: string
): Promise<{ maxOccupancy: number; minOccupancy: number } | null>
```

Les requêtes SQL ont été optimisées pour exploiter l'index :

```sql
WHERE h.measuring_set_id IN (:...measuringSetIds)
  AND h.record_datetime_utc BETWEEN :from AND :to
  AND h.data_points -> "$.occupancy" IS NOT NULL
```

Example 4.4. (Améliorations techniques):
- Utilisation de l'index de clé primaire
- Remplacement de `JSON_EXTRACT()` par l'opérateur `->` (plus lisible)
- Ajout de vérifications de nullité pour les tableaux vides
- Binding TypeORM d'arrays avec la syntaxe `:...array` pour les clauses `IN`

I.IV.3 - Injection de dépendances

Le pattern d'injection de dépendances a été correctement implémenté dans le conteneur IoC :

```
// app.module.ts
serviceContainer.bind<AttendanceStatsController>(AttendanceStatsController)
    .toSelf()
    .inSingletonScope();

// AttendanceStatsResolver.ts
@injectable()
export class AttendanceStatsResolver {
    constructor(private attendanceStatsController: AttendanceStatsController) {}
}
```

Cette approche améliore la testabilité et suit les patterns architecturaux existants du projet.

I.V - Résultats et impact

Gains de performance mesurés

Métrique	Avant	Après	Amélioration
Requête 30 jours	90+ secondes	40 ms	$\times 2\ 250$
Requête 1 an	Crash (timeout)	220 ms	$\infty \rightarrow 220$ ms
Opération DB	Full table scan	Index seek	—
Comportement	Dégradation exp.	Performance linéaire	—

I.VI - Explication des optimisations clés

I.VI.1 - Exploitation de l'index composite

L'index de clé primaire `(measuring_set_id, record_datetime_utc)` est désormais pleinement exploité :

Definition 4.5. (Stratégie d'indexation):
- `measuring_set_id IN (...)` restreint aux partitions pertinentes
- `record_datetime_utc BETWEEN` utilise la seconde partie de l'index composite
- MySQL peut ignorer complètement les données non pertinentes

I.VI.2 - Trade-off et analyse coût-bénéfice

- **Coût ajouté** : 1 appel API pour récupérer les measuring set IDs (10-20 ms)
- **Coût économisé** : Élimination du scan complet avec parsing JSON (90+ secondes)
- **Gain net** : Amélioration de performance de ×2 250

I.VII - Impact en production

Cette optimisation a permis de :

- Rendre les dashboards réactifs pour des statistiques annuelles en temps réel
- Réduire drastiquement la charge sur la base de données
- Éliminer les erreurs de timeout pour les requêtes multi-mois
- Améliorer significativement l'expérience utilisateur avec des réponses instantanées

I.VIII - Conformité aux patterns existants

L'optimisation suit le même pattern déjà utilisé avec succès dans :

- `app/modules/attendance/infrastructure/record-history-mysql.repository.ts`
- `app/modules/attendance/services/attendance.service.ts`

La solution réutilise l'infrastructure existante (`SensorsInternalHttpRepository`) et respecte le pattern repository utilisé dans l'ensemble du codebase.

I.IX - Enseignements techniques

Leçons clés

1. **Conscience des index** : Toujours concevoir les requêtes autour des index disponibles
2. **Prudence avec les colonnes JSON** : Le filtrage sur des champs JSON empêche l'utilisation d'index
3. **Requêtes en deux étapes** : Ajouter une étape de lookup légère peut être plus rapide qu'une requête unique non optimisée
4. **Mesurer systématiquement** : L'amélioration de ×2 250 a été validée par des mesures en production réelle
5. **Suivre les patterns existants** : La solution réutilise l'architecture établie du projet

Concepts techniques approfondis

- **Index composites** : Compréhension du fonctionnement de `(measuring_set_id, record_datetime_utc)`
- **TypeORM array binding** : Syntaxe `:...array` pour les clauses `IN`
- **Opérateurs JSON MySQL** : Utilisation de `->` au lieu de `JSON_EXTRACT()`
- **Injection de dépendances** : Patterns IoC pour améliorer testabilité et maintenabilité

Example 4.5. (Fichiers modifiés):

- `app/controllers/attendance-stats/AttendanceStatsController.ts` — Ajout DI et résolution measuring sets
- `app/repositories/AttendanceStatsRepository.ts` — Optimisation requêtes avec colonnes indexées
- `app/resolvers/AttendanceStatsResolver.ts` — Injection contrôleur au lieu d'appels statiques
- `app/app.module.ts` — Enregistrement du contrôleur dans le conteneur IoC

Date de réalisation : Octobre 2025

Statut : ✅ Déployé en production et validé avec du trafic réel